

AsymptoticAnalysis after eighteen reviews

A differential audit of native dispatch,
coefficient frontiers, and exact refinement

Repository: VladimirReshetnikov/Asymptotic

Snapshot: 6687962f3c858a4f93623cfc496f33e35c6763d4

Review date: 9 September 2026, Pacific time

Abstract

This report examines the current package against its existing review record rather than republishing that record. Its central conclusion is that the package’s strongest role is not merely to wrap Wolfram Language’s asymptotic functions: it is to preserve a selected real branch, an explicit sparse scale, transported remainder information, and replayable mathematical evidence. The newer native delegation layer broadens coverage, but its dispatch semantics must agree with the option interface it exposes.

Four current findings concern backend defaults, syntactic overprotection of nested pure functions, inconsistent option-name identity, and contradictory development-status documentation. Two additional findings materially sharpen existing review topics: the observable importer omits a native Taylor-endpoint check already present in the forward engine, and refinement can demand unavailable information from an ancestor even when the derived result is exactly constant. The former admits a source-level wrong-remainder construction with an honest but incomplete Taylor provider; the latter is both a completeness and a performance defect.

The archive includes candidate source patches, fifteen native regression candidates, a native characterization and benchmark harness, a novelty ledger, and forty-eight passing independent Python checks. **No native Wolfram package execution was obtained in this session.** Public-call outcomes are therefore distinguished from source-established control flow and independently checked mathematics. The patches are review candidates, not an accepted replacement release.

Evidence boundary	Source review and independent mathematical/contract models; native service unavailable.
Existing-review baseline	Maintained crosswalk: 18 reports and 123 finding entries.
Deliverable	Article source and PDF, code, reproducible independent results, and unrun native tests.

Contents

1	Executive assessment	3
2	Snapshot, scope, and non-duplication method	4
2.1	A moving repository requires a fixed reference	4
2.2	How the existing reports were used	4
2.3	Evidence classes	5
3	What the repository contributes	5
3.1	The representation is the main product	5
3.2	Special scales and supporting operations	6
3.3	Native delegation adds coverage, not an automatic proof upgrade	6
4	Comparison with built-in Wolfram Language	6
4.1	A fair capability matrix	6
4.2	The order-convention collision is real	7
4.3	An independent example of the sparse inverse advantage	8
5	New findings in the native interface	9
5.1	N01: The dispatcher ignores the advertised backend default	9
5.2	N02: A nested pure function is mistaken for a callable source	10
5.3	N03: Option classification disagrees with Wolfram option-name identity	11
5.4	N04: The README describes automatic routing as both implemented and still absent .	12
6	Sharper findings on mathematical information flow	12
6.1	D01: Observable composition can manufacture zero coefficients beyond a native endpoint	12
6.2	D02: Refinement can reject an exactly known constant because its ancestor cannot be refined	15
7	Trust boundaries that should not be weakened by these repairs	16
7.1	Formal residual, numerical comparison, and interval proof	16
7.2	Differentiation contracts are not optional decoration	17
7.3	Assumptions must survive the whole request	17
8	Performance analysis and a measurement plan	18
8.1	What can already be concluded	18
8.2	Benchmark design included in the archive	18
8.3	A request-local map is preferable to premature global caching	18

9	Proposed development sequence	19
9.1	First: make the current mathematical boundary safe	19
9.2	Second: make backend selection agree with the public interface	19
9.3	Third: turn compatibility into invariant-based tests	19
9.4	Fourth: share the coefficient-provider boundary	19
10	Artifacts, validation, and how to reproduce	20
10.1	Included files and their status	20
10.2	Independent execution obtained here	20
10.3	Native reproduction sequence	21
10.4	What these results do not establish	21
11	Conclusion	22
A	Compact novelty and evidence ledger	22
B	Repair acceptance checklist	23

1 Executive assessment

The most important new repair is the missing native-order guard in the observable path.

The code asks a Taylor provider for enough coefficients, but then consumes its returned coefficient list without checking that the returned series actually reaches that order. A missing coefficient *inside* a known series interval may mean zero; a coefficient *beyond* its recorded endpoint is unknown. Conflating these cases can turn a truthful $1 + t + \mathcal{O}(t^2)$ input into a false $1 + x + \mathcal{O}(x^4)$ output for a function whose true value is $1 + x + x^2$. This is a narrower, directly actionable extension of the existing native-tail review topic, not a claim to have newly discovered the general problem of native remainder semantics.

The most immediately visible new interface repair is honoring the declared backend default.

The package exposes "Backend" through `Options`, but its dispatcher hard-codes `Automatic` when an explicit selector is absent. Consequently, setting the advertised default to "Series" does not select that backend. This matters mathematically because the package's exclusive cutoff and native `Series`'s inclusive order need not retain the same coefficients.

The most useful new performance repair is to stop refinement at an already exact derived value.

A constant observable carries no remaining coefficient demand, even when its provenance mentions an uncertain inverse. Replaying that inverse is unnecessary, and can fail against a legitimate input-precision cap. The existing inverse-state implementation already has an exact finite-inverse fast path; the missing piece is the analogous treatment of exact derived representations.

The native-compatibility goal is valuable, but its success must be judged against three separate questions: Was the intended backend selected? Was its output preserved? Was any additional analytic contract justified? A successful answer to the second question does not automatically answer the other two. Conversely, refusing an analytic operation on a native result without a package remainder contract is a deliberate safety boundary, not a defect.

ID	Finding	Priority	Evidence and novelty
N01	Advertised backend defaults are ignored	Medium	Dispatcher inspection; post-review native layer; public regression supplied, unrun.
N02	Nested pure functions are mistaken for a callable-source contract	Medium	Exact guard trace; pure-redex and <code>Root</code> probes; no claim of a measured native failure.
N03	Held option parser uses structural identity instead of option-name identity	Medium	Source trace plus official <code>OptionValue</code> contract; public spelling-equivalence regressions supplied.
D01	Observable Taylor import lacks a returned-endpoint check	High	Source-level mathematical counterexample with a truthful custom provider; sharpens prior C06/C16.
D02	Exact derived refinement unnecessarily replays an uncertain ancestor	Medium	Source trace and exact dependency proof; sharpens prior C08/P07/P08, rather than repeating their general recommendations.
N04	README contradicts implemented automatic routing	Low	Direct contradiction between current README, implementation, and detailed compatibility document.

Priority describes the consequence of the issue, not a claim that every public reproduction has run. D01 has the highest mathematical consequence, but its concrete provider fixture is still awaiting native

execution. N01–N03 are chiefly routing and compatibility defects; they do not establish a newly unsound interval certificate.

2 Snapshot, scope, and non-duplication method

2.1 A moving repository requires a fixed reference

The audited commit is 6687962f3c858a4f93623cfc496f33e35c6763d4. Its recorded commit time is 10 September 2026 at 00:18:17 UTC, which is 9 September at 17:18:17 in the user’s Pacific timezone. The date change is a timezone conversion, not a claim to have reviewed a future local revision. All repository references in this article are pinned, rather than following `main` after the review.

At this snapshot the public package context is `AsymptoticAnalysis`; `AsymptoticInverse` remains a public function. The root single-file distribution is generated from modular kernel sources. The README declares version 1.8.0 and Wolfram Language 15.0 or later, and identifies upstream native evidence using 15.0.1 for Windows. Those are repository statements, not a kernel version measured in this session.^[1]

The inspected areas include the complete native-dispatch file; the arithmetic, observable, composition, differentiation, and refinement paths in `SeriesOperations.wl`; the rational certificate implementation; coordinate inversion and exact termination; selected core parser and jet routines; ordinary arithmetic normalization; inverse refinement state; focused native compatibility tests; and the relevant repository guides and review crosswalk. This was targeted source inspection through the connected repository service, not an automated full-checkout scan or an exhaustive execution of every adapter.

2.2 How the existing reports were used

The maintained implementation register and the two review indexes are the primary exclusion map. The register maps eighteen reports and 123 finding entries; its references identify the original snapshots and the individual evidence ledgers. Relevant original ledgers, including the contract-audit delta and the earlier structured findings, were also consulted where a proposed finding overlapped a recorded topic.^[2, 3, 4, 5, 6]

This article does not reissue the existing C01–C23, P01–P08, D01–D10, V01–V02, or X01–X12 lists as new findings. The labels D01 and D02 *in this article* mean differential findings; references to *prior C06*, *prior C08*, and so forth always refer to the repository register. The machine-readable novelty ledger uses distinct `AUDIT-D01` and `AUDIT-D02` identifiers to remove this possible ambiguity.

The newest native-dispatch file is absent at the second-wave review snapshot 921387e5ba1239bfda96e63e64e89bf63d9c41e6. This helps establish the chronology of N01–N03. The earlier reviews nevertheless contain broad compatibility recommendations, so the claimed contribution here is the specific current control-flow defect and its regression, not the general idea of improving compatibility.

D01 explicitly builds on the native-tail and opaque-function discussions. Its added content is an *analytic polynomial* with a truthful short Taylor provider, the precise missing check in `seriesJetApply`, and a comparison with the already guarded `fwdAnalytic`. D02 explicitly builds on refinement-demand concerns. Its added content is a zero-demand exact observable that fails despite needing no new coefficients, a connection to the existing exact-inverse fast path, and a conservative derived-value fast path.

No assertion is made that every sentence of all eighteen historical articles was reread. The novelty assessment is anchored to the maintained full crosswalk, the review indexes, the overlapping original

ledgers, and the inspected implementation. This is stronger than ignoring prior reports, but it is not an independent historical text-deduplication proof.

2.3 Evidence classes

The report separates four kinds of support. *Source-established* means that a displayed predicate, branch, or missing guard can be checked directly in the pinned code. *Mathematically established* means that the consequence follows from a proof independent of Wolfram evaluation. *Independently executed* means that the included Python model or symbolic calculation actually ran. *Native-pending* means that the supplied Wolfram program still needs to run against the actual package.

These classes are not interchangeable. In particular, an independent model of a dispatcher is not an implementation of Wolfram’s evaluator, and a fixture test of a text patch is not a package integration test. The Wolfram connector returned upstream/network errors even for a version query and $1 + 1$; no usable native kernel response was obtained. Direct container retrieval of a checkout also failed, so the patches were checked against retrieved source anchors and synthetic fixtures rather than applied to a locally cloned complete repository.

3 What the repository contributes

3.1 The representation is the main product

The ordinary analytic representation is essentially

$$A(x) + P(x) \left(\sum_{\beta < h} u(x)^\beta Q_\beta(\log u(x)) + \mathcal{O}\left(u(x)^\rho(1 + |\log u(x)|)^d\right) \right), \quad u(x) \rightarrow 0^+. \quad (1)$$

The local coordinate is positive on a selected real approach. Exponents can be exact real numbers; coefficient blocks are polynomials in a logarithmic marker; the prefactor and offset are exact. Stored assumptions, domains, recipes, and remainder metadata accompany the finite expression. The `GeneralizedSeries` head is therefore not merely pretty printing for a truncated formula.¹

A package cutoff acts on the recorded scale and is exclusive. The finite expression and its remainder must be interpreted together. `Normal` deliberately discards the latter. This is convenient for numerical experiments, but it is not a proof-preserving conversion to a fully known function. The code distinguishes native results from analytic series before allowing precision-sensitive operations.²³

The inverse engine normalizes models of the form

$$f(x) = y_0 + au^p \left(1 + \sum_j u^{\delta_j} B_j(\log u) \right), \quad (2)$$

with the necessary ordering, reality, and branch hypotheses. Direct Lagrange, grouped Lagrange, Newton, exact termination, and retained coefficient state provide different ways to construct or extend the same kind of information. The source does not lack Newton iteration or all incremental reuse; recommending those as entirely new features would repeat or contradict the existing implementation.⁴⁵

¹`src/Kernel/SeriesOperations.wl`, lines 1–145, at the pinned snapshot.

²`src/Kernel/AsymptoticAnalysis.wl`, lines 1100–1165, at the pinned snapshot.

³`src/Kernel/NativeCompatibility.wl`, lines 1–19, at the pinned snapshot.

⁴`src/Kernel/AsymptoticAnalysis.wl`, lines 935–1035, at the pinned snapshot.

⁵`src/Kernel/RefinementState.wl`, lines 1–170, at the pinned snapshot.

3.2 Special scales and supporting operations

The module `map` separates Lambert-style logarithmic and exponential cores, Gamma and Barnes inverse scales, source and target coordinate transformations, forward special-function adapters, finite flat sectors, Fourier-like coefficients, and composite envelopes. The intended benefit is to retain a useful asymptotic structure instead of forcing every intermediate result into one ordinary power-series container. The exact closure properties differ between these scales; the same head does not imply that every operation is supported on every kind.^[7, 1]

The ordinary arithmetic interface also has a held normalizer. Its rationale is substantive: evaluation of a reciprocal or an automatic cancellation can otherwise erase a denominator before the package has checked whether its leading term is known. The explicit calculus and the ordinary upvalue interface converge on precision-aware operations rather than on arithmetic with `Normal` alone.⁶

Refinement is intentionally not synonymous with truncation. Truncation can weaken known information. Refinement may replay the original source or an operation recipe, or reuse an inverse coefficient state. Differentiation requires more than a magnitude bound on an arbitrary remainder; the code has an explicit derivative-contract gate. These distinctions should be preserved while repairing the particular inconsistencies identified below.⁷

3.3 Native delegation adds coverage, not an automatic proof upgrade

The current wrapper supports explicit `"Series"` and `"Asymptotic"` backends and selected automatic routes. A native result records the native output, normalized expression, held request, original arguments, recognized specifications, kernel information, and a distinct remainder contract. It does not acquire `Exact -> True` simply because a native approximation lacks a visible O term. This is an appropriate design decision.⁸

The detailed compatibility document acknowledges that complete native-input coverage remains a goal rather than a completed guarantee. Automatic routing can be constrained by explicit package contracts; a representation failure can allow a selected fallback; an unresolved selected native backend does not imply that another backend was tried. These are existing documented boundaries, not additional discoveries in this report.^[8]

4 Comparison with built-in Wolfram Language

4.1 A fair capability matrix

A comparison must distinguish mathematical scope, output representation, and evidence strength. There is no basis here for a blanket claim that the package is faster than native functions, or that native functions cannot invert non-Taylor series.

⁶`src/Kernel/SeriesArithmetic.wl`, lines 1–180, at the pinned snapshot.

⁷`src/Kernel/SeriesOperations.wl`, lines 320–432, at the pinned snapshot.

⁸`src/Kernel/NativeCompatibility.wl`, lines 124–244, at the pinned snapshot.

Task	Built-in capabilities	Package position and recommended interpretation
Forward expansion	Series supports Taylor, Laurent, fractional-power and logarithmic behavior, symbolic coefficients, and successive variable specifications.[9]	Sparse exact-real exponent blocks and explicit real-branch remainder transport are the differentiator, not elementary Taylor coefficients. Native delegation handles other output shapes under a different contract.
Broad asymptotics	Asymptotic treats more than ordinary Taylor input, including special functions and other asymptotic scales.[10]	Specialized package engines retain their own structures; explicit native mode avoids pretending that all native forms have a package analytic envelope.
Series reversion	InverseSeries inverts suitable SeriesData ; documented examples include fractional-power inverses.[11]	The package focuses on real source branches, irrational power-log scales, transformed targets, and specialized inverse cores.
Implicit solutions	AsymptoticSolve handles equations and systems, branch points, real/complex centers, and several variables.[12]	A single-variable real inverse package is not a replacement for the full implicit-system solver. Direct equation-solving comparisons belong in a separate coverage matrix.
Arithmetic	SeriesData participates in arithmetic, composition, differentiation and integration.[13]	The package tracks logarithmic error degree and gates analytic operations by its stronger representation-specific contracts.
Numerical reference roots	FindRoot supplies numerical solutions with precision and accuracy controls.[14]	An asymptotic seed is useful, but a numerical reference comparison and an exact interval certificate are different outputs.
Discrete and operational asymptotics	The native ecosystem also includes asymptotic sums, integrals, differential and recurrence solvers, and discrete asymptotics.[15, 16]	Do not infer coverage of these operations merely from the package’s name or from the existence of a native result wrapper.
Options	Options , SetOptions , and OptionValue establish standard defaults and name-resolution conventions.[17, 18, 19]	N01 and N03 show that exposing an option is not enough: the held dispatcher must implement its semantics.

The current **Series** documentation lists **Analytic**, **Assumptions**, and **SeriesTermGoal**. In particular, **SeriesTermGoal** must not be described as unavailable to **Series** merely because older recollections of the option list omitted it. Likewise, **AsymptoticSolve**’s approximation order is not generally polynomial degree.[9, 12]

4.2 The order-convention collision is real

For the elementary source e^x , a native order-two power series has the quadratic term, whereas an exclusive package cutoff of two does not:

$$e^x = 1 + x + \frac{1}{2}x^2 + \mathcal{O}(x^3), \quad \text{native order 2,} \quad (3)$$

$$e^x = 1 + x + \mathcal{O}(x^2), \quad \text{exclusive cutoff 2.} \quad (4)$$

These identities follow from the exponential series. They are not claimed as newly executed package outputs. The different conventions are documented by the repository and native API.[8, 9]

Therefore, backend selection is part of the mathematical request. A dispatch error cannot be dismissed as a cosmetic metadata issue. Automatic delegation already records a native order convention; a future unified interface should preserve that distinction rather than silently treating the third positional argument as one universal notion of precision. The broad normalization problem is already in the review register; N01 isolates a new way in which the current implementation chooses the wrong convention despite a user-visible default.

4.3 An independent example of the sparse inverse advantage

Consider

$$f(x) = x + x^\alpha, \quad x > 0, \quad \alpha > 1. \quad (5)$$

The derivative is positive, so the small positive inverse is unique. Write

$$g(y) = yH(z), \quad z = y^{\alpha-1}. \quad (6)$$

Then H satisfies

$$H + zH^\alpha = 1, \quad H(0) = 1. \quad (7)$$

The implicit function theorem near $(H, z) = (1, 0)$ applies because the derivative with respect to H is one there. The real branch of H^α is analytic near $H = 1$. Thus H has a convergent local Taylor expansion in z , giving a rigorous asymptotic expansion in y .

Proposition 1. *For $n \geq 1$, the coefficient of z^n in H is*

$$c_n = \frac{(-1)^n}{n!} \prod_{j=0}^{n-2} (n\alpha - j), \quad c_0 = 1, \quad (8)$$

where the product is empty when $n = 1$.

Proof. Set $V = 1 - H$. Equation (7) becomes $V = z(1 - V)^\alpha$. Lagrange's coefficient identity gives

$$[z^n]V = \frac{1}{n} [t^{n-1}](1 - t)^{n\alpha} = \frac{(-1)^{n-1}}{n} \binom{n\alpha}{n-1}.$$

Negating this coefficient and expanding the binomial coefficient yields (8). The analyticity established above justifies the local expansion rather than only a formal manipulation. $\square$

The first terms are therefore

$$g(y) = y - y^\alpha + \alpha y^{2\alpha-1} - \frac{\alpha(3\alpha-1)}{2} y^{3\alpha-2} + \frac{\alpha(4\alpha-1)(2\alpha-1)}{3} y^{4\alpha-3} + \dots \quad (9)$$

After the term indexed by N , the remainder is $\mathcal{O}(y^{1+(N+1)(\alpha-1)})$. The included SymPy calculation independently substitutes the product-formula coefficients into (7) through degree seven and obtains zero for every residual coefficient checked. It does not call the repository.

For $\alpha = \sqrt{2}$, the exponent differences are irrational. A single conventional `SeriesData` with a rational exponent lattice and variable-independent coefficients cannot directly encode that infinite support. This is a representation observation, not a claim that every native Wolfram approach to the example fails: native output may use another shape, transformations, or an equation solver. The package's sparse support is nevertheless a natural direct representation for (9).[13]

5 New findings in the native interface

5.1 N01: The dispatcher ignores the advertised backend default

Classification: source-established interface defect; native regression pending.

Source and consequence

`NativeCompatibility.wl` adds `"Backend" -> Automatic` to `Options[AsymptoticExpansion]`. Its dispatcher collects explicit selector values and then uses

```
backend = If[values === {}, Automatic,
  ReleaseHold[First[values]]];
```

The no-selector branch does not consult the current option default. This remains true after `SetOptions` has changed that default.⁹¹⁰

The standard purpose of `SetOptions` is to change a symbol's default options. A fully specified `OptionValue[f, opts, name]` considers the explicit rules and the defaults of `f`; it is the appropriate mechanism for this request.^[18, 19]

The minimal public probe is:

```
saved = Options[AsymptoticExpansion];
SetOptions[AsymptoticExpansion, "Backend" -> "Series"];
s = AsymptoticExpansion[Exp[x], {x, 0, 2}];
{s["Kind"], Normal[s]}
Options[AsymptoticExpansion] = saved;
```

The source predicts automatic package selection, rather than the declared native default, for this ordinary exact-real input. The intended native result contains $x^2/2$; the package's exclusive-cutoff result does not. The supplied test uses cleanup that restores the previous defaults even when evaluation is interrupted.

Repair and acceptance conditions

The minimal candidate replaces the hard-coded fallback with

```
OptionValue[AsymptoticExpansion, {}, "Backend"]
```

while retaining the first explicit selector when one is present. Invalid effective values must still reach the existing `InvalidBackend` failure. Delayed defaults should be materialized once per dispatch decision, not once for selection and again for metadata.

The alias requires an explicit policy. It currently delegates to the canonical entry and receives a copied option list at load time. The candidate patch makes the canonical defaults effective; it does *not* implement independent mutable defaults on `AsymptoticExpand`. Either document that the alias shares the canonical defaults, or pass the public entry symbol through the held dispatcher so that each entry can resolve its own defaults. Silently exposing two independently changeable option lists while reading neither correctly is not a useful policy.

⁹[src/Kernel/NativeCompatibility.wl](#), lines 5–10, at the pinned snapshot.

¹⁰[src/Kernel/NativeCompatibility.wl](#), lines 63–81, at the pinned snapshot.

Acceptance tests should cover an altered default, an explicit overriding selector, a delayed default with an evaluation counter, and restoration of ordinary automatic behavior. A single explicit-backend example cannot expose this defect because it never takes the faulty branch.

Novelty

This finding is about the newly implemented held dispatcher, not a repetition of the older cutoff-normalization recommendation. Its distinguishing observation is that the default shown by `Options` and the backend actually used can disagree on the same request.

5.2 N02: A nested pure function is mistaken for a callable source

Classification: source-established overbroad admission guard; public outcomes pending.

The protection predicate searches the entire held source for `_Function`. A match forces package-only handling, even if that function is not the source callable but a nested part of an ordinary expression. The prepared-source guard repeats the same broad test.¹¹

Two mathematically equivalent, side-effect-free sources illustrate the issue:

```
AsymptoticExpansion[I + x, {x, 0, 2}]
AsymptoticExpansion[Function[t, t][I] + x, {x, 0, 2}]
```

The second source contains a reducible pure-function application. It is not a callable supplied in place of the source expression. Nonetheless, the original held tree contains `Function`, so it enters the protected package path before evaluation can reduce the expression to $I + x$. The first spelling is eligible for representation fallback after the real analytic engine rejects the nonreal coefficient; the second is not.

The same structural issue can affect a `Root` coefficient, whose defining polynomial is encoded by a pure function. For example, consider `Root[1 + ## + ##^4 &, 1]`. Its polynomial has no real zero: the minimum of $t^4 + t + 1$ on $\mathbb{R}$ is

$$1 - \frac{3}{4^{4/3}} > 0.$$

Such a coefficient belongs naturally in a native complex result. A pure function inside its algebraic representation should not, by itself, assert a package callable-source contract. This is a motivated regression candidate, not an assertion that a particular native `Root` simplification was observed here.

Repair: classify the role, not just the node type

The candidate checks `Function` at the root of the held source, and recognizes the prepared callable wrapper at that root. It keeps recursive protection for actual inverse-function syntax, conditional sources, existing series objects, and remainder objects, and keeps explicit branch, direction, and resource contracts binding.

This is not a proposal to evaluate arbitrary user programs early just to make dispatch convenient. The original source remains held and retained as provenance. The narrower check removes a syntactic false positive without discarding the separate contracts that justify package-only handling. Its regression must include both the reducible expression and an actual source callable; making both delegate would be an incorrect repair.

¹¹[src/Kernel/NativeCompatibility.wl](#), lines 115–123, at the pinned snapshot.

Novelty

The existing compatibility plan already says that not every native-supported input is automatically covered. N02 adds a specific, generalizable reason: a representation detail nested in an expression is mistaken for a semantic role of the whole source. The relevant acceptance invariant is stability under a side-effect-free identity redex, not an unqualified promise of arbitrary evaluator equivalence.

5.3 N03: Option classification disagrees with Wolfram option-name identity

Classification: source-established parser inconsistency; native regressions pending.

The dispatcher extracts held option keys and compares them structurally with lists such as `HoldComplete[Assumptions]` and `HoldComplete["MaxTerms"]`. The backend selector itself matches only the literal string `"Backend"`. Meanwhile, the package's eventual `OptionValue` calls follow Wolfram's option-name semantics.¹²

The official `OptionValue` documentation specifies that contexts are ignored for option-name comparison and that a symbol and its symbol-name string are interchangeable. Structural equality of the held syntax does not implement that equivalence.^[19]

A focused example avoids any ambiguity about which native engine to choose:

```
AsymptoticExpansion[Sqrt[a^2] Exp[x], {x, 0, 2},
  "Assumptions" -> a > 0, "Backend" -> "Package"]
```

The string key `"Assumptions"` does not equal the stored held symbol key. It is classified as a native-only extra and rejected before the ordinary option reader can consume it. Using the symbol `Assumptions` takes a different path. A symbolic spelling of `Backend` can be missed by the selector and, depending on its position and form, treated as an option conflict or an expansion specification rather than a selector.

One identity relation must be used throughout dispatch

The candidate introduces a held option-name function. A held string maps to that string; a held symbol maps to its unevaluated `SymbolName`. Known option names are then mapped back to their canonical declared keys. The original rule and delayed value remain held for delegation.

This normalization must be used by selector extraction, selector stripping, specification exclusion, package-only contract checks, and request-key collection. Fixing only one of these leaves a second inconsistent parser in the same call path. Unknown names must remain unknown; normalization is not permission to drop unrecognized options.

The resulting map should be request-local or derived from current defaults. Caching it globally without invalidation can reintroduce N01 when `SetOptions` changes a public default or an option set. Equally important, a key's own value must not be evaluated simply to discover its option name. The candidate uses `SymbolName[Unevaluated[name]]` for that reason.

Remaining grammar question

A symbol can be both a possible expansion variable and a familiar option name. A fully general native-input parser should use positional grammar and the selected backend's accepted specification

¹²[src/Kernel/NativeCompatibility.wl](#), lines 31–113, at the pinned snapshot.

forms, not only name membership, to distinguish those roles. That larger grammar redesign is not completed by the candidate patch. The present repair addresses equivalent spellings of actual option rules; it does not claim to solve every ambiguity in a native input-superset interface.

5.4 N04: The README describes automatic routing as both implemented and still absent

Classification: source-established documentation contradiction.

The detailed compatibility document and current implementation describe selected automatic native routes and representation fallback. The README's development-status paragraph nevertheless says that `Automatic` uses the same package path as explicit `"Package"`, and describes automatic fallback as still required. The paragraph also foregrounds an older explicit-native test total without clearly separating it from later automatic-routing evidence.¹³[8]

The supplied replacement states the actual three-way distinction: automatic mode retains successful package results and performs selected delegation; package mode requires the analytic engines; explicit native modes choose native semantics. It points to the detailed document for snapshot-specific evidence rather than making one historical test count stand in for the current implementation.

This is not a cosmetic disagreement. Readers use the README to decide whether a call preserves package precision and branch requirements, while maintainers use it to interpret what a reported test total validates. The correction should land with the routing fixes, not be deferred until a broader documentation rewrite.

6 Sharper findings on mathematical information flow

6.1 D01: Observable composition can manufacture zero coefficients beyond a native endpoint

Classification: source-level wrong-remainder construction; native fixture pending; delta to prior C06/C16.

The omitted check

The generic unary branch of `seriesJetApply` computes a Taylor order from the requested output cutoff and the valuation of the inner increment. It asks `Series` for that order, checks that the result is a regular `SeriesData` with suitable coefficient independence, and defines a coefficient accessor that returns zero when an index falls outside the stored list. It does *not* check that the native endpoint is beyond the requested order.¹⁴

The corresponding forward-engine path, `fwdAnalytic`, already includes the stronger test

```
! less[N0, s[[5]]/s[[6]]]
```

¹³README.md, lines 130–201, at the pinned snapshot.

¹⁴src/Kernel/SeriesOperations.wl, lines 236–290, at the pinned snapshot.

as a reason not to consume that Taylor result through the fast analytic route. The observable path has duplicated a closely related conversion without duplicating this guard. This comparison is more informative than a general statement that native imports deserve scrutiny.¹⁵

An honest short provider, not a lying oracle

Let the observable be the polynomial

$$h(t) = 1 + t + t^2.$$

Suppose its specialized Taylor provider supplies

$$\text{SeriesData}[t, 0, \{1, 1\}, 0, 2, 1], \quad (10)$$

that is, $1 + t + \mathcal{O}(t^2)$, when asked for order four. This is incomplete but truthful. It does not assert that the quadratic coefficient is zero. A user-defined symbolic function is permitted to supply a conservative partial result; an importer cannot reinterpret that result as a more complete one solely because it had requested more information.

The supplied fixture gives the function its polynomial value at numeric arguments and a custom `Series` upvalue returning (10). A separate native regression first checks that the fixture actually establishes the intended native result. If that precondition fails on a particular kernel, the subsequent public-path observation is not evidence of this counterexample. This explicit precondition avoids treating a guessed evaluator interaction as an observed result.

Now take the exact inner series $s(x) = x$ and request

```
s = AsymptoticExpansion[x, {x, 0, 5}, "Backend" -> "Package"];
r = SeriesObservable[s, ShortTaylor[z], z, "Cutoff" -> 4];
```

where `ShortTaylor` is the supplied fixture. The following derivation is a source trace, independent of whether that fixture has yet run in a native kernel.

The inner positive jet has valuation one, exact input precision, and logarithmic degree zero. The requested Taylor probe has order four. The returned data in (10) pass the existing shape, regularity, denominator, and coefficient-independence checks. The coefficient accessor provides $c_1 = 1$ and substitutes $c_2 = c_3 = 0$. The unit-series routine therefore retains only x below cutoff four. Its precision calculation uses the exact inner precision and requested cutoff, not the returned native endpoint, so it assigns remainder order four. Adding the observable's constant value yields

$$1 + x + \mathcal{O}(x^4). \quad (11)$$

The true observable is $1 + x + x^2$.

Proposition 2. *Under the truthful provider contract (10), the current observable conversion can assign the false remainder (11).*

Proof. The source trace above gives the retained expression and the assigned remainder. Their error relative to $h(x)$ is x^2 . Since $x^2/x^4 = x^{-2}$ is unbounded as $x \rightarrow 0^+$, this error is not $\mathcal{O}(x^4)$. All coefficients and inputs in the example are real; the source function is an analytic polynomial. Neither complex-coefficient rejection nor an opaque-nonanalytic-function caveat removes the information loss. $\square$

The independent Python model reproduces the zero-filling decision and checks the exact error ratio at $x = 2^{-k}$, where it equals 4^k . This is corroboration of the mathematics and the modeled conversion, not a native package run.

¹⁵`src/Kernel/AsymptoticAnalysis.wl`, lines 478–529, at the pinned snapshot.

Why this is a substantive delta

Prior C06 concerns the relationship between native formal order and analytic logarithmic tails, and prior C16 concerns unproved analyticity of opaque functions. The present example assumes a truthful ordinary Taylor bound and an analytic polynomial. The failure is that a *recorded low endpoint is not enforced at all* on this public operation route. It also identifies an implementation asymmetry: a relevant endpoint guard has already been added to a neighboring forward path.

This does not establish that a stock built-in currently returns (10) for a larger order request. No such stock-function witness is claimed. The narrower statement is sufficient for an importer defect: the accepted representation contains less information than the conversion subsequently uses, and an honest extension provider makes the mismatch explicit.

Minimal repair and its limits

The supplied patch adds a fail-closed endpoint check before constructing the coefficient accessor:

```
If[! less[n, native[[5]]/native[[6]]],
  fail["InsufficientNativeOrder",
    "The observable Taylor probe does not cover the required order.",
    <|"RequiredCoefficientOrder" -> n,
    "NativeEndpoint" -> native[[5]]/native[[6]]|>]];
```

The packaged version also preserves the returned native result in the diagnostic. This is conservative and intentionally mirrors the existing forward-engine request/endpoint convention. It may refuse a shortened probe that would still suffice for some retained coefficients. Recovering such cases requires a precision-aware import, not merely removing the guard.

For a more complete importer, suppose the native tail is genuinely $\mathcal{O}(t^m)$ and the inner increment obeys

$$\eta(x) = \mathcal{O}\left(u^\alpha(1 + |\log u|)^d\right), \quad \alpha > 0.$$

Then the transported native tail is bounded by

$$\mathcal{O}\left(u^{\alpha m}(1 + |\log u|)^{dm}\right). \tag{12}$$

The output precision must intersect that information with the uncertainty already present in the inner argument. In the polynomial counterexample, this would retain the truthful $\mathcal{O}(x^2)$ bound instead of rejecting the operation.

Equation (12) assumes the native tail premise; **SeriesData** syntax alone does not prove that premise for every opaque source. Thus the candidate fixes an order-completeness hole, not the entire pre-existing analyticity-admission problem. The two obligations should remain separate in code and documentation.

A coefficient oracle needs three states

A useful local abstraction distinguishes: a stored coefficient; an omitted zero *strictly inside* a known coefficient interval; and an unknown coefficient at or beyond its endpoint. The endpoint is not a synonym for the length of the coefficient array. A short array may omit interior zeros while still carrying a later known endpoint. Conversely, an early endpoint makes later coefficients unknown even if the requested order was large.

Exact termination is a fourth, stronger piece of evidence, not something inferred from a missing stored coefficient. The existing core already uses a distinguished terminator in some coefficient generators; extending that idea to native import would reduce the risk of another zero/unknown conflation. This proposal is a concrete implementation of the information boundary exposed by D01, not a new claim that the repository lacks all precision-aware arithmetic.

6.2 D02: Refinement can reject an exactly known constant because its ancestor cannot be refined

Classification: source-established unnecessary dependency; native public regression pending; delta to prior C08.

A public construction with no malformed object

Start with an inverse whose source is declared only to a finite asymptotic precision:

```
s = AsymptoticInverse[x + x^2, {x, 0}, {y, 3},
  "InputRemainder" -> {4, 0}];
c = SeriesObservable[s, 7, z];
SeriesRefine[c, 20]
```

The inverse describes information compatible with a source $x + x^2 + \mathcal{O}(x^4)$; requesting substantially more inverse information than the transported cap is appropriately refused. But the observable $z \mapsto 7$ is exactly constant for every input in that family. Its output is 7 with zero remainder, and needs no further inverse information.

The ordinary observable path constructs precisely such an exact derived representation. Nevertheless, `SeriesRefine` obtains its recipe, recursively refines its operands first, and only afterwards switches on the operation kind. For this positive-valuation ancestor the requested operand cutoff is $h + 2$. At $h = 20$, the ancestor is asked for cutoff 22 and rejects that request against the declared forward-precision cap. The failure is returned before the constant observable is recomputed.¹⁶¹⁷

The inverse-state helper does not rescue this case: its first gate requires an ordinary inverse kind and model, whereas the constant is a derived object. That helper does already recognize exact finite inverse results, making the asymmetry particularly clear.¹⁸

The demand is exactly zero

Proposition 3. *If an observable H is a known constant on the retained domain, refining $H \circ g$ requires no additional coefficients of g , regardless of the precision cap attached to g .*

Proof. For any two admissible inputs g_1, g_2 , one has $H(g_1) = H(g_2) = c$. Consequently the output is determined without distinguishing any members of the input family. No increase in input precision can change the finite expression or reduce an already zero output remainder. $\square$

¹⁶[src/Kernel/SeriesOperations.wl](#), lines 270–294, at the pinned snapshot.

¹⁷[src/Kernel/SeriesOperations.wl](#), lines 370–432, at the pinned snapshot.

¹⁸[src/Kernel/RefinementState.wl](#), lines 133–169, at the pinned snapshot.

The claim does not erase the stored domain or make an undefined constant composition valid outside its domain. It says that, on the domain already admitted by the constructor, the source's uncertain coefficients are no longer needed to determine the output.

This extends the earlier demand-planning concern in a different direction. A negative power can require *more* source precision than a fixed margin provides. A constant observable can require *none*. A planner that gives every operand a positive demand is incomplete even if its positive demands are made arbitrarily generous.

A conservative exact-value fast path

The candidate patch recognizes an ordinary derived representation with an infinite precision frontier and a requested cutoff that retains every stored block. It validates the resource option, preserves the exact prefactor, offset, assumptions, domain, expression, and provenance, updates the requested-cutoff metadata, and returns without replaying the source. It records "Strategy" -> "ExactValueNoOp" and zero new coefficient evaluations.

The restriction on retained blocks is important. A request at or below an existing block exponent may invoke the package's current lower-cutoff retargeting behavior. That separate policy, already discussed in the review register, is left unchanged. Native results, specialized representations without an ordinary cached series representation, and uncertain derived jets do not take the fast path.

A general backward-demand planner remains useful, but it is not necessary to fix this case. The local exact-value rule is small, independently understandable, and consistent with the existing exact finite-inverse optimization. It also avoids claiming that a successful no-op has replayed an operation recipe; its statistics explicitly say otherwise.

Performance consequence without invented timings

For the example above, the requested cutoff can grow without bound while the output remains exactly 7. The legacy dependency plan makes an ancestor request and can fail before producing that answer. The repaired plan makes zero ancestor coefficient requests. This is an information-flow improvement independent of any wall-clock benchmark.

It is not a claim of constant-time serialization of the entire result object, nor a measurement of Wolfram memory use. Reading a large retained representation or writing its provenance can still have a cost. The guarantee is narrower and useful: requested output order no longer drives mathematically irrelevant coefficient generation.

7 Trust boundaries that should not be weakened by these repairs

7.1 Formal residual, numerical comparison, and interval proof

The inspected residual code explicitly describes composition with the finite forward model. Such a residual can verify coefficient cancellation without certifying the original function's unknown tail. The numerical checker uses a reference root and reports a comparison, not an interval theorem. The certificate code constructs rational enclosures for supported elementary expressions and states the function and

interval to which its proof applies. These are different evidence products, not three interchangeable names for validation.¹⁹²⁰

The native `FindRoot` API provides numerical root finding with configurable precision and accuracy goals. It is an appropriate source of reference values, but the package’s interval certificate has additional mathematical obligations not satisfied merely by a small numerical residual.^[14]

A basic certificate argument explains the distinction. Let F be continuously differentiable on a real interval I , and suppose

$$0 < \mu \leq |F'(x)| \quad (x \in I)$$

with derivative sign fixed. At a center $c \in I$, suppose $|F(c) - y| \leq \varepsilon$. If the interval

$$J = [c - \varepsilon/\mu, c + \varepsilon/\mu]$$

is contained in I , the mean-value theorem gives opposite endpoint residual signs, hence existence of a root in J by continuity; the derivative condition gives uniqueness in I . A sharper interval-Newton correction can then intersect enclosures using the derivative interval. This is a proof about F , y , and I , rather than an asymptotic statement about a family of source germs.

The current implementation includes source-side checks, source-domain checks, derivative separation, residual enclosures, and rational interval refinement. Inspection of those paths did not yield a new certificate-soundness counterexample. That is not a proof of the entire implementation, and it is not a reason to erase the still-open issues already listed in the repository. It is a reason not to manufacture a new certificate finding merely to lengthen this report.

7.2 Differentiation contracts are not optional decoration

A value estimate $r(u) = \mathcal{O}(u^p)$ does not by itself imply $r'(u) = \mathcal{O}(u^{p-1})$. For example,

$$r(u) = u^{p+1} \sin(u^{-3})$$

is $\mathcal{O}(u^{p+1})$ and hence $\mathcal{O}(u^p)$ near zero, but its derivative contains $-3u^{p-3} \cos(u^{-3})$. Consequently, allowing native or formal data into the analytic derivative path just because a finite expression can be differentiated would discard a real proof obligation.

The package’s derivative-order gate addresses this distinction. None of N01–N03 should remove that gate. N02 narrows a false-positive callable classification; it does not authorize analytic arithmetic on arbitrary native output. D01 validates available Taylor order; it does not supply derivative bounds for every imported formal remainder.²¹

7.3 Assumptions must survive the whole request

The core’s scoped assumption machinery and native delayed-option handling are deliberate parts of the current design. The proposed option-name normalization must not reevaluate a delayed assumption to obtain a second metadata copy, nor move source evaluation outside the selected backend’s context. Its job is only to identify an option name consistently.

For the same reason, the narrow callable fix must retain the original held request even when a side-effect-free expression reduces to a simpler prepared source. Semantic normalization for routing does not justify inventing a new provenance history. The original request, effective option values, selected backend, and returned contract are distinct pieces of information.

¹⁹ `src/Kernel/AsymptoticAnalysis.wl`, lines 1174–1249, at the pinned snapshot.

²⁰ `src/Kernel/InverseCertificates.wl`, lines 1–450, at the pinned snapshot.

²¹ `src/Kernel/SeriesOperations.wl`, lines 324–355, at the pinned snapshot.

8 Performance analysis and a measurement plan

8.1 What can already be concluded

This session produced no native timing or memory measurements. In particular, the independent test runtime in `independent_checks.txt` is the runtime of a small Python model suite, not a benchmark of the repository.

Three structural conclusions are justified. First, exact derived no-ops can avoid all ancestor coefficient generation, as proved in D02. Second, request-local option canonicalization needs only the supplied keys and the current declared option set; it does not require evaluating a source or running a symbolic simplifier. Third, the native wrapper stores both a native result and its normalization, so a timing comparison with a direct native call must account for the additional requested output work. Treating those two workloads as identical would be misleading.

The last point is not a newly discovered memory leak. The prior reviews already discuss retained representations and provenance cost. Here it specifies how to measure the *new dispatch changes* without misattributing deliberate normalization cost to a regression.

8.2 Benchmark design included in the archive

The supplied native harness has four variants: direct **Series**, explicit native wrapper, automatic wrapper, and exact derived refinement. It records a first call in the process, then repeated warm batches, kernel version, system identifier, selected entry point, result head, and memory-in-use differences at the end of each batch. It deliberately does not label those differences as peak memory.

Each variant should run in a separate fresh kernel against the baseline and staged candidate. The exact-refinement workload needs a semantic column: the baseline’s rapid failure and the candidate’s successful exact result are not equal successful workloads, so a raw speed ratio would be meaningless. For the wrapper workloads, compare native result preservation and contract metadata before comparing elapsed time.

There should be three classes of input. Small stable native series expose dispatch overhead. Medium expansions expose normalization and result-storage overhead. Large symbolic coefficient cases expose whether key handling or repeated source preparation accidentally triggers additional expensive evaluation. A source with deliberate side effects should be used only as an evaluation-count test, not as a benchmark whose result is assumed mathematically interchangeable under reordering.

8.3 A request-local map is preferable to premature global caching

Let k be the number of supplied option rules and p the size of the declared option list. The simple candidate’s repeated lookup costs on the order of kp elementary name comparisons. A request-local map can reduce repeated lookup work after one pass through the option list. That refinement should be considered only after the native semantic tests pass.

A process-global cache needs an invalidation rule when defaults or option declarations change. N01 is precisely a default-resolution problem, so introducing a cache that freezes the default map would trade one inconsistency for another. The desired invariant is not “use a cache” but “resolve the current effective request once, without rereading delayed values.”

9 Proposed development sequence

9.1 First: make the current mathematical boundary safe

Land D01 as a small fail-closed change with its honest short-provider regression and a sufficient-provider control. Review any other path that turns a native coefficient list into a coefficient oracle against the same three-state distinction. The immediate acceptance condition is that an unavailable coefficient cannot be reported as known zero. Broader analytic admission remains a separately tracked obligation, not something this patch can declare solved.

Land D02 independently. Its acceptance conditions are exact constant refinement beyond an ancestor's cap, unchanged treatment of uncertain outputs, preservation of domains and prefactors, validation of invalid budgets, and no change to lower-cutoff cases that discard retained blocks. Record the actual no-op in refinement statistics rather than pretending that more coefficient work occurred.

9.2 Second: make backend selection agree with the public interface

N01 should be tested before and after every dispatch refactor because it detects whether the declared default is actually read. N03 should then establish a single held key-identity function used at every classification boundary. N02 should narrow only the false-positive pure-function test, with controls for actual callables and nested inverse-function expressions.

N04 should be corrected in the same development sequence. Documentation must describe the behavior that the tests are meant to validate. An automatic-routing feature described simultaneously as implemented and not yet implemented prevents an otherwise useful review ledger from functioning as a reliable user guide.

9.3 Third: turn compatibility into invariant-based tests

The native-interface test suite already contains many valuable direct-backend controls. The next additions should test relations between requests, rather than merely collecting more unrelated examples:

1. An explicit selector and the same effective declared default should select the same backend and convention.
2. Equivalent option-name spellings should be classified the same way without reevaluating the values.
3. A side-effect-free reducible function application inside an expression should not assert a callable-source contract for the whole expression.
4. Increasing the requested order may reveal unknown coefficients, but may not turn them into known zeros.
5. Exact derived values should not acquire a positive coefficient demand solely because their provenance mentions an uncertain ancestor.

These invariants are the incremental development opportunity identified here. They are more targeted than another general recommendation for more tests, and they can be exercised with a small deterministic matrix before testing more expensive special-function cases.

9.4 Fourth: share the coefficient-provider boundary

The immediate code duplication is between ordinary forward analytic conversion and observable conversion. A small shared admission function should answer whether a returned native object has the

correct variable, center, regularity, lattice, coefficient independence, and usable endpoint. It should return structured information, not a Boolean that causes downstream code to guess what was established. This is a limited refactor. It need not begin with a universal transseries framework, a new proof kernel, or a complete rewrite of every special adapter. Those much broader ambitions already appear in the repository’s research and engineering backlog. The measurable local deliverable is that two callers no longer disagree about what the same Taylor data permit them to read.

10 Artifacts, validation, and how to reproduce

10.1 Included files and their status

Artifact	Purpose and status
<code>article/audit.tex</code> , <code>audit.pdf</code>	This article and its compiled, visually checked PDF.
<code>code/patch_snapshot.py</code>	Stages twelve exact-anchor substitutions in three files, in a new directory. Candidate only; not natively integrated.
<code>code/fixtures.wl</code>	Honest incomplete Taylor provider used by the D01 tests. Its native establishment is a separate test precondition.
<code>code/native_regressions.wlt</code>	Fifteen desired-behavior tests, including positive controls. Unrun here; several should fail on the baseline.
<code>code/characterize.wls</code>	Records native observations, messages, expression/remainder summaries, and environment when run by the user. Unrun here.
<code>code/benchmark_native.wls</code>	Records separate-process first and warm-batch measurements for four variants. Unrun here.
<code>code/audit_models.py</code> and tests	Independent option, coefficient-frontier, exact-dependency, and mathematical models. Executed; not a Wolfram evaluator.
<code>evidence/independent_validation.json</code>	Forty-eight successful Python test methods, with environment and test identifiers.
<code>evidence/novelty_ledger.json</code>	Six findings, prior-review relationships, evidence status, and candidate-repair scope.
<code>evidence/review_scope.json</code>	Inspected areas, cited snapshots, and limitations of this review.

The patcher validates all expected text anchors before creating its output. It refuses an existing output directory and normally requires the pinned Git revision. It does not modify the source checkout, push a branch, or copy an out-of-date standalone distribution into the staged result. The `-no-git-check` switch is for a separately identified archive; it does not disable the source-anchor checks.

10.2 Independent execution obtained here

Run the supplied model checks with

```
python code/run_independent.py
```

They require Python and SymPy. The final evidence records 48 successful test methods, zero failures, and zero errors. Some methods contain multiple deterministic subcases, which are deliberately not inflated

into the advertised method count. The checks cover the modeled default branch, name equivalence, role-sensitive protection, zero versus unknown coefficients, the false remainder ratio, exact-value demand elimination, generalized inverse coefficients through order seven, and fail-closed text-patch behavior.

An initial fixture assertion contained a closing-delimiter typo; it was corrected before the final successful run. No native baseline or candidate failure was hidden by that correction: the affected assertion inspected a generated source string, and no native package run took place at any stage.

The synthetic patch fixtures verify replacement mechanics, not the completeness of a local checkout or correctness of Wolfram evaluation. Source anchors were separately compared with the connector-returned pinned ranges. Before integration, the candidate must still be staged against a real checkout and run in a native kernel.

10.3 Native reproduction sequence

From a checkout at the pinned revision, stage the candidates in a separate path:

```
python code/patch_snapshot.py --repo /path/to/Asymptotic \
  --out /path/to/asymptotic-audit-candidate
```

Then characterize the baseline and candidate in separate native processes:

```
wolframscript -file code/characterize.wls \
  /path/to/Asymptotic/src/Kernel/AsymptoticAnalysis.wl \
  baseline-native.json

wolframscript -file code/characterize.wls \
  /path/to/asymptotic-audit-candidate/src/Kernel/AsymptoticAnalysis.wl \
  candidate-native.json
```

In a fresh Wolfram kernel, load the selected modular entry point and run

```
Get["/path/to/src/Kernel/AsymptoticAnalysis.wl"];
TestReport["/path/to/audit/code/native_regressions.wlt"]
```

The fixture-establishment test for D01 must pass before interpreting that public-path counterexample. Review the current upstream focused tests for native compatibility, observables, assumption scoping, arithmetic, and refinement as well; the fifteen new tests are intentionally not a replacement for those tests.

After accepting changes into the repository, regenerate the standalone distribution with the repository's own build process and run its loading/freshness checks. The audit staging directory does not claim to be a finished release artifact. The existing distribution and validation backlog is not repeated here as a newly discovered finding.

10.4 What these results do not establish

There is no measured native speedup, no native pass count for the candidate, no assertion of complete support for all native argument forms, and no proof that the package's entire analytic or interval implementation is sound. The included evidence is nevertheless useful: it identifies exact current source decisions, supplies mathematical counterexamples where appropriate, proposes localized changes, and gives the next evaluator a reproducible way to confirm or reject each public-path prediction.

11 Conclusion

The repository already contains substantial mathematics, specialized scale handling, precision-aware arithmetic, retained inverse state, and a serious effort to keep different evidence products distinct. Its recent evolution also shows why a differential audit must inspect the latest source instead of repeating an earlier review’s list of missing features.

The central implementation lesson is to preserve information boundaries at every transition. The backend must come from the effective public request, not a hard-coded fallback. An option name must mean the same thing before and after held dispatch. A nested pure function must not silently change the role of an entire source expression. A returned coefficient frontier must bound what a consumer reads, even when the requested order was larger. An exact result must not be made dependent again on information it no longer needs.

The immediate repair sequence is small enough to review independently: enforce the observable endpoint; add the exact derived-value fast path; correct backend defaults and option-name handling; narrow the pure-function guard; and align the README with the implemented behavior. Native execution remains the necessary next validation step, not a result asserted by this article.

A Compact novelty and evidence ledger

Audit ID	Relationship to prior work	Added information and limitation
N01	Native compatibility goals; no matching current defect in the maintained crosswalk	Hard-coded missing-selector branch ignores mutable declared defaults; coefficient-retention consequence and focused regression. No native observation.
N02	Existing protected-source policy and incomplete-coverage goal	Whole-tree Function search misclassifies applied redexes and Root polynomial encodings; root-role candidate guard. No stock-kernel coverage claim.
N03	Existing option/UX normalization topics	Held structural keys disagree with documented symbol/string/context option-name equivalence; shared canonical-key candidate across five parser responsibilities. Remaining specification grammar is not solved.
D01	Prior C06 and C16	Missing returned-endpoint guard in observable conversion; truthful analytic-polynomial short provider; false $\mathcal{O}(x^4)$ proof; comparison with guarded forward route. Native fixture unrun.
D02	Prior C08, with related P07/P08	Exact constant with precision-capped ancestor; proof of zero demand; existing inverse-only fast path contrasted with derived replay; new derived-value no-op. Native public regression unrun.
N04	Documentation and compatibility tracking	Specific present-tense contradiction in current README and an exact replacement paragraph. Not a general documentation-overhaul recommendation.

B Repair acceptance checklist

The staged patch should not be accepted solely because it applies. Confirm that explicit backend selection still preserves native results and delayed-option behavior; altered canonical defaults are honored; original protected branch and budget contracts are retained; genuine source callables still enter their supported package route; option spelling normalization does not evaluate keys or values; short native Taylor data cannot supply unknown coefficients; adequate Taylor providers still work; exact derived no-ops preserve metadata and domains; uncertain series are not promoted to exact ones; and lower-cutoff behavior remains unchanged when known blocks would be removed.

The independence of these conditions matters. A patch can repair N02 by delegating too much, or repair D01 by returning a safe failure for every observable. Positive controls are therefore as important as the original counterexample. The supplied native suite includes both, but their status remains unrun until a native kernel records otherwise.

References

- [1] Vladimir Reshetnikov, *Asymptotic repository README*, pinned snapshot 6687962f3c85. <https://github.com/VladimirReshetnikov/Asymptotic/blob/6687962f3c85a4f93623cfc496f33e35c6763d4/README.md>.
- [2] *Code review status and implementation register*, same snapshot. https://github.com/VladimirReshetnikov/Asymptotic/blob/6687962f3c85a4f93623cfc496f33e35c6763d4/docs/development/CODE_REVIEW_STATUS.md.
- [3] *External code-review index*, same snapshot. <https://github.com/VladimirReshetnikov/Asymptotic/blob/6687962f3c85a4f93623cfc496f33e35c6763d4/external-reports/code-review/README.md>.
- [4] *Second-wave review index*, same snapshot. <https://github.com/VladimirReshetnikov/Asymptotic/blob/6687962f3c85a4f93623cfc496f33e35c6763d4/external-reports/code-review/wave-2/README.md>.
- [5] *Review 11 findings delta*, original review snapshot 921387e5ba1239bfda96e63e64e89bf63d9c41e6; retrieved from the current pinned archive. <https://github.com/VladimirReshetnikov/Asymptotic/blob/6687962f3c85a4f93623cfc496f33e35c6763d4/external-reports/code-review/wave-2/code-review-11/evidence/findings-delta.json>.
- [6] *Review 4 structured findings*, retrieved from the current pinned archive. <https://github.com/VladimirReshetnikov/Asymptotic/blob/6687962f3c85a4f93623cfc496f33e35c6763d4/external-reports/code-review/wave-1/code-review-4/evidence/findings.json>.
- [7] *Kernel module map*, current pinned snapshot. <https://github.com/VladimirReshetnikov/Asymptotic/blob/6687962f3c85a4f93623cfc496f33e35c6763d4/src/Kernel/README.md>.
- [8] *Native compatibility plan and evidence*, current pinned snapshot. https://github.com/VladimirReshetnikov/Asymptotic/blob/6687962f3c85a4f93623cfc496f33e35c6763d4/docs/development/NATIVE_COMPATIBILITY.md.
- [9] Wolfram Research, *Series*, Wolfram Language documentation, accessed 9 September 2026, Pacific time. <https://reference.wolfram.com/language/ref/Series.html>.

- [10] Wolfram Research, *Asymptotic*, official documentation, accessed on the same date. <https://reference.wolfram.com/language/ref/Asymptotic.html>.
- [11] Wolfram Research, *InverseSeries*, official documentation. <https://reference.wolfram.com/language/ref/InverseSeries.html>.
- [12] Wolfram Research, *AsymptoticSolve*, official documentation. <https://reference.wolfram.com/language/ref/AsymptoticSolve.html>.
- [13] Wolfram Research, *SeriesData*, official documentation. <https://reference.wolfram.com/language/ref/SeriesData.html>.
- [14] Wolfram Research, *FindRoot*, official documentation. <https://reference.wolfram.com/language/ref/FindRoot.html>.
- [15] Wolfram Research, *Asymptotics*, official guide. <https://reference.wolframcloud.com/language/guide/Asymptotics.html>.
- [16] Wolfram Research, *DiscreteAsymptotic*, official documentation. <https://reference.wolfram.com/language/ref/DiscreteAsymptotic.html>.
- [17] Wolfram Research, *Options*, official documentation. <https://reference.wolfram.com/language/ref/Options.html>.
- [18] Wolfram Research, *SetOptions*, official documentation. <https://reference.wolfram.com/language/ref/SetOptions.html>.
- [19] Wolfram Research, *OptionValue*, official documentation. <https://reference.wolfram.com/language/ref/OptionValue.html>.